OPTIMIZACIÓN, DOCUMENTACIÓN Y CONTROL DE VERSIONES

David Campos Garrido y Diego Vargas Delgado

Grupo 5, Proyecto 8

FormValidator.java | .gitignore

```java
package Grupo5;

public class FormValidator {

    public boolean validar(String nombre, String email, String telefono,
            String direccion, String ciudad, String codigoPostal) {

        if (nombre == null || nombre.isEmpty()) return false;
        if (email == null || !email.contains("@")) return false;
        if (telefono == null || telefono.length() < 9) return false;
        if (direccion == null || direccion.isEmpty()) return false;
        if (ciudad == null || ciudad.isEmpty()) return false;
        if (codigoPostal == null || codigoPostal.length() != 5) return false;
        return true;
    }
}
```

Código a refactorizar

Primera refactorización

En la primera modificación hemos refactorizado de la siguiente forma:

FormValidator.java

```java
package Grupo5;

public class FormValidator {

    final int TAMAÑOTEL = 9;
    final int TAMAÑOCP= 5;

    public boolean validar(String nombre, String email, String telefono,
            String direccion, String ciudad, String codigoPostal) {

        if (nombre == null || nombre.isEmpty()) return false;
        if (email == null || !email.contains("@")) return false;
        if (telefono == null || telefono.length() < TAMAÑOTEL) return false;
        if (direccion == null || direccion.isEmpty()) return false;
        if (ciudad == null || ciudad.isEmpty()) return false;
        if (codigoPostal == null || codigoPostal.length() != TAMAÑOCP ) return false;
        return true;
    }
}
```

En el tercer y sexto if existían valores numéricos que hemos decidido convertir en constantes dado que siempre deben ser esos valores.

Segunda refactorización

```java
FormValidator.java
package Grupo5;

public class FormValidator {

    final int TAMAÑOTEL = 9;
    final int TAMAÑOCP= 5;

    public boolean validar(ValidarParameter parameterObject) {

        if (parameterObject.nombre == null || parameterObject.nombre.isEmpty()) return false;
        if (parameterObject.email == null || !parameterObject.email.contains("@")) return false;
        if (parameterObject.telefono == null || parameterObject.telefono.length() < TAMAÑOTEL) return false;
        if (parameterObject.direccion == null || parameterObject.direccion.isEmpty()) return false;
        if (parameterObject.ciudad == null || parameterObject.ciudad.isEmpty()) return false;
        if (parameterObject.codigoPostal == null || parameterObject.codigoPostal.length() != TAMAÑOCP ) return false;
        return true;


    }
}
```

Para llegar a este punto usamos “Introduce Parameter Object…” en el método validar.

Esto se hace porque el código original tenía 6 String, con el código nuevo tenemos un única clase que los contenga.

Tercera Refactorización

```java
*FormValidator.java
package Grupo5;

public class FormValidator {

    public boolean validar(ValidarParameter parameterObject) {

        if (parameterObject.nombre == null || parameterObject.nombre.isEmpty()) return false;
        if (isInvalidEmail(parameterObject)) return false;
        if (isInvalidTelefono(parameterObject)) return false;
        if (parameterObject.direccion == null || parameterObject.direccion.isEmpty()) return false;
        if (parameterObject.ciudad == null || parameterObject.ciudad.isEmpty()) return false;
        if (isInvalidCodigoPostal(parameterObject)) return false;
        return true;
    }

    private boolean isInvalidCodigoPostal(ValidarParameter parameterObject) {
        return parameterObject.codigoPostal == null || parameterObject.codigoPostal.length() != 5;
    }

    private boolean isInvalidTelefono(ValidarParameter parameterObject) {
        return parameterObject.telefono == null || parameterObject.telefono.length() < 9;
    }

    private boolean isInvalidEmail(ValidarParameter parameterObject) {
        return parameterObject.email == null || !parameterObject.email.contains("@");
    }
}
```

En las condiciones de email, telefono y codigoPostal usamos un “Extract Method” de la sección Refactor que nos proporciona Eclipse, viéndose más claras las reglas que pide el código.